

5.13.82

Rushil Shanmukha Srinivas
EE25BTECH11057
Electrical Engineering ,
IIT Hyderabad.

October 10, 2025

1 Problem

2 Solution

- Latin Square and Permutations

3 C Code

4 Python Code

Problem Statement

Question : Let α and β be the distinct roots of the equation $x^2 + x - 1 = 0$. Consider the set $T = \{1, \alpha, \beta\}$. For a 3×3 matrix

$\mathbf{M} = (a_{ij})_{3 \times 3}$, define $R_i = a_{i1} + a_{i2} + a_{i3}$,

$C_j = a_{1j} + a_{2j} + a_{3j}$, for $i, j = 1, 2, 3$. Match the following:

- (A) The number of matrices $\mathbf{M} = (a_{ij})_{3 \times 3}$ with all entries in T such that $R_i = C_j = 0$ for all i, j , is
- (B) The number of symmetric matrices $\mathbf{M} = (a_{ij})_{3 \times 3}$ with all entries in T such that $C_j = 0$ for all j , is
- (C) Let $\mathbf{M} = (a_{ij})_{3 \times 3}$ be a matrix with all entries in T such that $R_i = 0$ for all i . Then the absolute value of the determinant of $\mathbf{M}$ is

① 1

③ infinite

⑤ 0

② 12

④ 6

The correct option is:

① $A \rightarrow 4, B \rightarrow 2, C \rightarrow 1$

③ $A \rightarrow 2, B \rightarrow 4, C \rightarrow 3$

② $A \rightarrow 2, B \rightarrow 4, C \rightarrow 5$

④ $A \rightarrow 1, B \rightarrow 5, C \rightarrow 4$

Latin Square and Permutations

Let $T = \{1, \alpha, \beta\}$ where α, β are distinct roots of $x^2 + x - 1 = 0$. For a 3×3 matrix $M = (a_{ij})$, define

$$R_i = a_{i1} + a_{i2} + a_{i3}, \quad C_j = a_{1j} + a_{2j} + a_{3j}. \quad (3.1)$$

$$\alpha + \beta = \frac{-\text{coefficient of } x}{\text{coefficient of } x^2} = -1 \quad (3.2)$$

$$1 + \alpha + \beta = 0 \quad (3.3)$$

(A) We need to find matrices **M** with all entries in T such that $R_i = C_j = 0$ for all i, j , So

$$R_1 = R_2 = R_3 = C_1 = C_2 = C_3 = 0. \quad (3.4)$$

Only possible case is some permutation of $\{1, \alpha, \beta\}$ as $1 + \alpha + \beta = 0$.
As both row total = column total = 0.

Latin Square :

Latin Square is a Matrix in which one element occurs exactly once in a row and a column.

The Matrix $\mathbf{M}$ should be Latin Square of order 3 on the symbol set $\{1, \alpha, \beta\}$

By permutations , we get

$$3 \times 2 \times 1 \times 2 = 12 \quad (3.5)$$

One of the 12 Latin Square Matrices is given as

$$\mathbf{M} = \begin{pmatrix} 1 & \alpha & \beta \\ \alpha & \beta & 1 \\ \beta & 1 & \alpha \end{pmatrix} \quad (3.6)$$

The number of Matrices $\mathbf{M} = 12$.

Thus, $A = 2$.

(B) Symmetric matrices $\mathbf{M}$ with entries in T such that $C_j = 0$.

$$\mathbf{M} = \mathbf{M}^T \quad (3.7)$$

So both $R_i = C_j = 0$ for all i, j .

Latin Square :

So The Matrix $\mathbf{M}$ should be Latin Square of order 3 on the symbol set $\{1, \alpha, \beta\}$ and satisfying $\mathbf{M} = \mathbf{M}^T$.

By permutations, we get

$$3 \times 2 \times 1 = 6 \quad (3.8)$$

One of the 6 Latin Square Matrices is given as

$$\mathbf{M} = \begin{pmatrix} 1 & \alpha & \beta \\ \alpha & \beta & 1 \\ \beta & 1 & \alpha \end{pmatrix} \quad (3.9)$$

The number of Matrices $\mathbf{M} = 6$.

Thus, $B = 4$.

(C) Matrices $\mathbf{M}$ with entries in T such that $R_i = 0$ for all i . So each row is a permutation of $(1, \alpha, \beta)$.

$$1 + \alpha + \beta = 0 \quad (3.10)$$

If rows are identical

$$|\mathbf{M}| = 0 \quad (3.11)$$

If rows are distinct then

For matrix $\mathbf{M}$ if $C_1 \rightarrow C_1 + C_2 + C_3$ then every element of C_1 is 0.

So

$$|\mathbf{M}| = 0. \quad (3.12)$$

Thus $C = 5$.

$$\boxed{A = 2, B = 4, C = 5} \quad (3.13)$$

Correct option is (b).

C Code

```
#include <stdio.h>
#include <math.h>
#define N 3
// Roots of  $x^2 + x - 1 = 0$ 
//  $\alpha = (-1 + \sqrt{5})/2$ 
//  $\beta = (-1 - \sqrt{5})/2$ 

// We'll store possible elements {1, alpha, beta}
void get_elements(double T[3]) {
    double alpha = (-1 + sqrt(5.0)) / 2.0;
    double beta = (-1 - sqrt(5.0)) / 2.0;
    T[0] = 1.0;
    T[1] = alpha;
    T[2] = beta;
}

// Check if sum of array of length 3 = 0 (within tolerance)
int is_zero_sum(double a, double b, double c) {
    double sum = a + b + c;
```

```

    return fabs(sum) < 1e-6;
}

// Compute determinant of 3x3 matrix
double determinant(double M[3][3]) {
    double det =
        M[0][0]*(M[1][1]*M[2][2] - M[1][2]*M[2][1]) -
        M[0][1]*(M[1][0]*M[2][2] - M[1][2]*M[2][0]) +
        M[0][2]*(M[1][0]*M[2][1] - M[1][1]*M[2][0]);
    return det;
}

int main() {
    double T[3];
    get_elements(T);

    int countA = 0; // For part (A)
    int countB = 0; // For part (B)
    double detVal = 0; // For part (C)
    int foundC = 0;

```

```

// --- (A): all entries in T such that  $R_i = C_j = 0$  ---
for (int a11=0;a11<3;a11++)
for (int a12=0;a12<3;a12++)
for (int a13=0;a13<3;a13++)
for (int a21=0;a21<3;a21++)
for (int a22=0;a22<3;a22++)
for (int a23=0;a23<3;a23++)
for (int a31=0;a31<3;a31++)
for (int a32=0;a32<3;a32++)
for (int a33=0;a33<3;a33++) {
    double M[3][3] = {
        {T[a11], T[a12], T[a13]},
        {T[a21], T[a22], T[a23]},
        {T[a31], T[a32], T[a33]}
    };
    // Row sums
    double R1 = M[0][0] + M[0][1] + M[0][2];
    double R2 = M[1][0] + M[1][1] + M[1][2];
    double R3 = M[2][0] + M[2][1] + M[2][2];
    // Column sums
    double C1 = M[0][0] + M[1][0] + M[2][0];
    double C2 = M[0][1] + M[1][1] + M[2][1];
    double C3 = M[0][2] + M[1][2] + M[2][2];
}

```

```

    double C2 = M[0][1] + M[1][1] + M[2][1];
    double C3 = M[0][2] + M[1][2] + M[2][2];
    if (fabs(C1)<1e-6 && fabs(C2)<1e-6 && fabs(C3)<1e-6)
        countB++;
}

// --- (C): any matrix with all Ri=0; compute |det(M)| ---
for (int a11=0;a11<3;a11++)
for (int a12=0;a12<3;a12++)
for (int a13=0;a13<3;a13++)
for (int a21=0;a21<3;a21++)
for (int a22=0;a22<3;a22++)
for (int a23=0;a23<3;a23++)
for (int a31=0;a31<3;a31++)
for (int a32=0;a32<3;a32++)
for (int a33=0;a33<3;a33++) {
    double M[3][3] = {
        {T[a11], T[a12], T[a13]},
        {T[a21], T[a22], T[a23]},
        {T[a31], T[a32], T[a33]}
    };
};

```

```

double R1 = M[0][0] + M[0][1] + M[0][2];
double R2 = M[1][0] + M[1][1] + M[1][2];
double R3 = M[2][0] + M[2][1] + M[2][2];
if (fabs(R1)<1e-6 && fabs(R2)<1e-6 && fabs(R3)<1e-6) {
    double det = determinant(M);
    detVal = fabs(det);
    foundC = 1;
    break;
}
}
printf("Computed results:\n");
printf("(A) Number of matrices (R_i=C_j=0): %d\n", countA);
printf("(B) Number of symmetric matrices (C_j=0): %d\n",
    countB);
if (foundC)
    printf("(C) |det(M)| for such a matrix: %.0f\n", detVal);

printf("\nHence, A2, B4, C3 (Correct Option = 3)\n");
return 0;
}

```

```
import ctypes
import os

so_file = "./matrix.so"

lib = ctypes.CDLL(so_file)

# The C code has main() printing everything,
# so we can just call it directly.
# But we could also expose a solve function if needed.

print("Running matrix problem solver (C code)...\n")

# Call C's main() function
lib.main()
```